

Python Introduction

Python is a scripting language - much like BASH / SHELL is a list of commands.

We run Python programs using a program called `python`. There are multiple versions of python, version 3 is current standard and version I will focus on.

[Python](#) is one of many scripting languages. Others include [Perl](#), [Ruby](#), [Rust](#) and even the Bash/Shell programming we've been talking about.

It is a script because we write the code in a text file and then run it directly with an interpreter. Before when we ran a script in bash/shell we would do

```
bash myscript.sh
```

To run code for python we will do

```
python myscript.py
```

A first script

Here's a text file that is called 'hello.py' and contains the following. This would be the content of a text file we called `hello.py`.

```
print("Hello World!")
```

```
python hello.py
```

```
Hello World!
```

```
python hello.py > message.txt # can redirect the output  
cat message.txt
```

```
Hello World!
```

The Python interpreter

The Python interpreter is a program we run, giving it a file or script which specifies the actions for it to take.

One can also just run the interpreter on the command line and interact with it directly. Just execute the python command on the command line.

```
$ python
```

```
Python 3.12.4 (main, Jun 6 2024, 18:26:44) [Clang 15.0.0 (clang-1500.3.9.4)] on darwin  
Type "help", "copyright", "credits" or "license" for more information.
```

```
>>> print("hello world")
```

```
hello world
```

Other math capabilities

The `%` operator is modulus - it means return the remainder after dividing by the number. It is useful to see if something divides evenly into a number

```

>>> 10 / 3
3.3333333333333335
>>> 10 % 3  # 10 / 3 is 3 with remainder of 1
1
>>> 10 / 2
5.0
>>> 10 % 2   # 10 / 2 is 5 with a remainder of 0
0

```

This can be useful to see if something is “Modulo” something (or Mod) – really useful when looking at sequence data and checking to see if it will translate – is the length/value mod 3 == 0 - means it is divisible perfectly by 3.

Strings in Python

Quotes can define the different special characters and use of strings

```

>>> 'break dance'
'break dance'
>>> "break dance"
'break dance'
>>> "don't break dance"
"don't break dance"
>>> 'don\'t break dance'
"don't break dance"
>>> ('concatenate ' 'these ' 'together')
'concatenate these together'

```

Variables

Data can be assigned to variables. Variables don't exist until they are declared.

The = is used for assignment. Some modifiers allow for modifying variable in place

- += add to the value of the variable
- -=, /=, *= are other in-place modifiers

```

>>> d = 0 # declare the variable d
>>> print("d is ", d)
d is 0
>>> print("e is ", e)
Traceback (most recent call last):
  File "<stdin>", line 1, in <module>
NameError: name 'e' is not defined
>>> d=8*7
>>> print(d)
56

```

```
>>> d += 100
>>> print(d)
156
```

String functions

Documented well in the python [string](#) library code.

- `string.find(substring[, start[, end]])`

Find a substring within a string

```
>>> str = "Jumping cow over the moon"
>>> str.find("cow")
8
>>> str.find("o") # start searching from beginning
9
>>> str.find("o",12) # start searching at position 12
12
```

- `string.split(separator, max_split)`

Split a string into a list based on a separator (great for column delimited data!).
`max_splits` specifies if it should stop after N separators are found

```
>>> str = "Jumping cow over the moon"
>>> str.split(" ")
['Jumping', 'cow', 'over', 'the', 'moon']
>>> str.split(" ",2)
['Jumping', 'cow', 'over the moon']
```

substring - extracting parts of a larger string

You can query the string with the `[ ]` operator to get a subset of the string

```
>>> msg="I am a golden god!"
>>> msg[:4] # everything before position 4
'I am'
>>> msg[7:13] # get a middle part from 7-13
'golden'
>>> msg[14:] # get from 14 to the end
'god!'
>>> msg[msg.find('am'):msg.find('!')]
'am a golden god'
```

Lists and list functions

These are useful for collecting items you can enumerate.

```
>>> l = [ 2, 3, 5]
>>> l[0] # lists start at 0
2
>>> l[1:3]
[3, 5]
>>> l.append(10)
>>> print(l)
[2, 3, 5, 10]
```

The `sum()` function will summate a list (add all the numbers up), `len()` will report how long it is. `max()` will report the largest number in a list

```
sum(l)
# prints
# 20
len(l)
# prints
# 4
max(l)
# prints
# 10
```

Sorting lists

```
l = [10,3,17,4]
l.sort()
l
# prints
#[3, 4, 10, 17]
ls = ['zf','fz','no','apple']
ls.sort()
print(ls)
# prints
#[ 'apple', 'fz', 'no', 'zf']
```

Using split to parse strings into lists

Starting with strings containing several parts which encode information you want.

```
Symbol:NASDAQ=AAPL;Date:2015-10-01;Performance:Open=111.29,Close=109.56
Symbol:NASDAQ=MSFT;Date:2015-10-01;Performance:Open=46.65,Close=46.53
```

Extract some of the info using `split`

```
q = "Symbol:NASDAQ=AAPL;Date:2015-10-01;Performance:Open=111.29,Close=109.56"
types = q.split(";")
print(types)
```

```

# will print
# ['Symbol:NASDAQ=AAPL', 'Date:2015-10-01',
#  'Performance:Open=111.29,Close=109.56']
symbolset = types[0].split(":")
print(symbolset)
# will print
# ['Symbol', 'NASDAQ=AAPL']
symbol = symbolset[1].split("=")
print(symbol)
# will print
# ['NASDAQ', 'AAPL']
print(symbol[1])
# will print
# AAPL
print(symbolset[1].split("=")[1])
# will print
# AAPL

```

repr function to display an object/string/number

```

# use repr to print out literally the string
hello = 'hello world\n'
print(hello)
# would print
# hello world

print(repr(hello))
# would print
# 'hello world\n'

```

More fancy printing

Can use formatting to print some things out like these important [numbers and phrase](#).

A formatted string uses the % operator to specify placeholder. Formatted printing is also easy if you are comfortable with style used in most programming languages.

```

n = [1, 2, 'oh my god']
print(n)
# prints
# [1, 2, 'oh my god']
# print things out with fancier printing
print("%s %s %s."%(n[0],n[1],n[2]))
# prints
# 1 2 oh my god.

```

```

print("%5s %5s %-10s"%(n[0],n[1],n[2]))
# prints
#      1      2 oh my god .
print("%-5s %5s %-10s"%(n[0],n[1],n[2]))
# prints
# 1      2 oh my god .
print("%-5s %5s %20s"%(n[0],n[1],n[2]))
# prints
# 1          2          oh my god.

```

Print out with placeholders

The format is first a **formatting** string which contains symbols {}. These {} are replaced, in order, by the items that follow in the `format()` function applied.

```

a = 15
b = 41
print( "a + b = ", a+b)
# will print
# a + b = 56

print( "{} + {} = {}".format(a,b,a+b))
# will print
# 15 + 41 = 56
print("{} ({} ) + {} ({} ) = {}".format("a",a,"b",b,a+b))
# will print
# a (15) + b (41) = 56

```

Format strings with rjust and ljust

This is less typical usage.

Can also use `rjust` and `ljust` to right or left justify a string in place.

```

n = [1, 2, 'oh my god']
print(n)
# will print
# [1, 2, 'oh my god']
# print things out with fancier printing
print(n[0].rjust(5), n[1].rjust(5), n[2].ljust(10))
# gives error
# AttributeError: 'int' object has no attribute 'rjust'

print( repr(n[0]).rjust(5), repr(n[1]).rjust(5), n[2].ljust(10))
#      1      2 oh my god
print( repr(n[0]).ljust(5), repr(n[1]).rjust(5), n[2].ljust(10))
#1          2 oh my god

```

```

print( repr(n[0]).ljust(5), repr(n[1]).rjust(5), n[2].rjust(10))
#1      2 oh my god
print( repr(n[0]).rjust(5), repr(n[1]).rjust(5), n[2].rjust(12))
# 1      2 oh my god
print( repr(n[0]).rjust(5), repr(n[1]).rjust(5), n[2].ljust(12))
# 1      2 oh my god

```

String joining and appending

Strings can be appended with the + operator.

```

left = "Sun"
right = "Fish"

print(left + "-" + right)

```

```

right = "Shine"
print(left + "-" + right)

```

The * operator followed by a number will just repeat a string that number of times.

```

print("Do you want to volunteer for the mosquito trial?: ", "No "*10)

```

Can be useful for generating runs of single letters eg.

```

seq1="AAGAGTCA"
seq2="TTGATAG"
print(seq1 + "N"*100 + seq2)

```

Case study

You can make strings upper or lower case with `string.upper()` and `string.lower()`.

```

bases="GGATAGaattaNGGT"
print(bases.upper())
print(bases.lower())

```

Practice processing strings

Let's initialize some data and process it.

```

# make a string
codestr= "17,20,30,12,5,6,19,13"
# split it into an array
dat=codestr.split(",")
# print the first item
print(dat[0])

```

```

# print the sum of the 2nd and 3rd items
# this won't work because they are strings
#print(dat[1]+dat[2])
print(int(dat[1]) + int(dat[2]) )
# print the summation of the whole dataset
# this won't work because it is an array of strings
#print(sum(dat))
intarray = [ int(x) for x in dat]
print("total sum is: ",sum(intarray))

# make a new array from the 3rd and 6th columns
newarray=[dat[2],dat[5]]
# print the new array
print("the new array is ",newarray)

# make a new string from the array contents
# sort the array
intarraysort = sorted(intarray)
# convert items to a string and then join together
print(", ".join([str(x) for x in intarraysort]))

```